
L2 Market

Microstructure System

Complete project report

From a bar backtester to deterministic passive-execution research and a verified C++ order book

Pranav Pillai
National University of Singapore

12 September 2026

Corrected V3 research reference: 04df629 Protocol locked: 63f4f0b

github.com/pranavpillaiNUS/l2-mm-system

Research question. Does a signal that predicts future mid-price movement remain useful after passive execution selects which states become fills?

Principal result. Normalized one-second order-flow imbalance has a positive coefficient in all 24 development windows, with pooled effect 0.123285 bps per signal standard deviation. Its locked 30-second passive-fill screen fails at both queue-credit endpoints. No candidate advances; the 12-window holdout remains strategy-sealed.

Engineering result. The corrected Python model makes arrival timing, delayed cancellations, FIFO queue competition, risk, and data recovery explicit. The C++17 order book matches the Python reference across every state of a recorded development hour. This report distinguishes native book-update measurements from complete replay performance.

Binance spot BTCUSDT; April–May 2026 development data. Offline research and simulation. Raw captures are excluded from Git; selected derived artifacts, data identities, native parity evidence, and report source are published. The earlier August report remains the historical V2 record.

Abstract

This report describes the complete research and engineering path of an offline market-microstructure system. An educational hourly bar backtester supplied interfaces and testing lessons, but its performance claims were demoted after execution and annualization limitations were identified. The project then recorded Binance spot depth and aggregate trades, reconstructed an exact L2 book, modeled passive orders and queue position, and studied the difference between price prediction and fill-conditioned economics.

The corrected V3 experiment reruns the same 24 five-hour development windows under explicit private-event scheduling and a post-response-gated snapshot policy. Each window aggregates five independently reset hourly episodes. The microprice baseline has mean net P&L of -0.774 and -0.978 USDT per window at zero and proportional queue-cancellation credit, respectively. Normalized OFI retains a positive population association at one second, with HAC $t = 64.6214$ and $R^2 = 0.0365623$. The predeclared 30-second selected-bucket separation is -0.456 and -0.064 bps, below the 1.0 bps materiality screen at both endpoints. Diagnostic window-clustered intervals cross zero. The study therefore blocks the proposed OFI-gated candidate and preserves the strategy-sealed holdout; it does not establish that all OFI strategies are ineffective.

A C++17 implementation uses checked scaled integers and reproduces the exact Python Decimal book's canonical bytes and state-hash transcript. The recorded-hour parity test covers 36,000 states. Preloaded book-update benchmarks show a 13.74497-fold ratio of Python to C++ median duration on the recorded hour. Parser, execution, research accounting, language-boundary costs, and full replay require separate measurement. The contribution is a traceable combination of execution-aware research, negative development evidence, a deterministic behavioral reference, and a measured native implementation, with limitations and reproduction steps made explicit.

Contents

1	Purpose, Scope, and Project Evolution	1
1.1	What the project is trying to establish	1
1.2	The bar-backtesting precursor	1
1.3	Why the project moved to L2	1
1.4	Model corrections and the meaning of completion	2
2	Data, Reconstruction, and System Architecture	2
2.1	Recorded data and its clock fields	2
2.2	Snapshot bridging and fail-closed recovery	3
2.3	Integrity inventory and selected panel	3
2.4	Software responsibilities and flow	4
3	Deterministic Execution and Risk	4
3.1	The execution model defines the fill sample	4
3.2	Clock and order lifecycle	5
3.3	External queue credit and own-order FIFO	5
3.4	Worst-case working exposure	6
3.5	Accounting, marks, and termination	6
4	Locked Research Design	7
4.1	Price, microprice, and order-flow imbalance	7
4.2	Frozen baseline and episode construction	8
4.3	Prospective hypotheses and decision rule	8
4.4	Uncertainty and the unit of inference	8
4.5	Holdout boundary	9

5	Corrected V3 Results and Historical Context	9
5.1	Baseline economics under the corrected model	9
5.2	Population OFI survives the correction	10
5.3	The passive-fill screen fails	10
5.4	What changed relative to historical V2	12
5.5	What the evidence answers	12
6	C++ Design, Integration, and Performance	12
6.1	Porting against a behavioral reference	12
6.2	Checked fixed-point representation	12
6.3	The Python/native boundary	13
6.4	Layered parity evidence	13
6.5	Isolated book-update measurement	14
6.6	Integrated timing has a different denominator	15
7	Verification, Failure Analysis, and Limits	15
7.1	What verification establishes	15
7.2	Failures that changed the project	16
7.3	Empirical and inferential limits	17
7.4	Execution and operational limits	17
7.5	Completed outcome and useful extensions	18
A	Reproduction and Artifact Map	19
A.1	Three distinct reproduction levels	19
A.2	Install and verify mechanics	19
A.3	Verify V3 without raw captures	19
A.4	Reproduce the frozen research run	20
A.5	Use and validate the native backend	20
A.6	Build the complete report	20
A.7	Canonical identities	21
A.8	Artifact map	21

1 Purpose, Scope, and Project Evolution

1.1 What the project is trying to establish

A passive limit order does not receive a random sample of future price moves. It fills when opposing flow consumes the modeled quantity ahead of it, while other orders may remain unfilled or be cancelled. A signal can therefore predict the market's next mid-price move without improving the outcome of the particular trades a passive maker receives. The project asks whether that distinction is visible in a controlled BTCUSDT experiment.

The deliverable is an offline chain from recorded data to a documented research decision: capture and integrity inventory; sequence-valid book reconstruction; deterministic replay; explicit execution and inventory constraints; economic and signal diagnostics; a predeclared development gate; and verified artifacts. Native implementation follows a behavioral contract so that speed can be assessed after correctness. A profitable strategy is an empirical possibility, not a requirement imposed on the answer.

This is a small-order counterfactual simulator over aggregate L2. It is not a market-by-order reconstruction, a calibrated exchange replica, or a live trading service. The distinction matters because an exact implementation of a queue assumption does not make that assumption observable in the underlying data.

1.2 The bar-backtesting precursor

The initial framework used BTCUSDT hourly OHLCV data and a common interface for momentum, mean reversion, moving-average crossover, and Bollinger-band strategies. Data loading, transaction costs, portfolio accounting, metrics, parameter sweeps, and walk-forward orchestration were separate components. Useful lessons included recreating stateful strategies between runs and checking whether fees erased a gross result.

The historical Phase 1 ratios are not current strategy evidence. The audit found same-close signal observation and execution, hourly equity annualized as 365 periods per year, an “out-of-sample Sharpe” formed from monthly Sharpe estimates, and an optimizer source overwritten during plotting. The metrics helper now infers observation frequency, and the logged search grids have a reconstructed runner; the old CSVs retain their historical identity. These limitations motivated the move to observable book events rather than a more elaborate interpretation of coarse bar returns.

1.3 Why the project moved to L2

Bars cannot identify post-only eligibility at arrival, displayed volume ahead, partial fills, cancellation races, or whether a fill is followed by an adverse move. L2 depth and trades expose enough information to model these mechanisms, while still leaving true queue position latent. The engineering design makes that missing information an explicit parameter, rather than silently treating a price touch as a guaranteed fill.

Initial L2 exploration rejected attractive results contaminated by unintended taker execution. A later profitable passive anchor did not generalize across the expanded panel. The fixed microprice baseline and OFI premise test then became a structured development study, with a reserved holdout and a stopping rule when the premise failed.

Table 1: The project record preserves distinct research and implementation stages.

Stage	Deliverable	Evidential role
Jan–Apr 2026	Bar framework and walk-forward exercise	Educational precursor; historical performance limitations are retained.
Apr–May 2026	L2 capture, replay, baseline exploration	Established passive execution mechanics and the development hypothesis.
Jun 2026	Frozen V2 24-window study	Historical baseline, OFI gate, queue-credit and latency stress under <code>legacy_book_update_v1</code> .
Jul–Aug 2026	Execution and publication audits	Exact private scheduling, own FIFO, working exposure, snapshot timing, and explicit model boundaries.
Sep 2026	Corrected V3 development rerun	Same development data and locked baseline; current-model economics and diagnostic conditional uncertainty.
Sep 2026	Native implementation and report closure	Checked C++ book, Python parity, scoped measurement, and this complete project account.

1.4 Model corrections and the meaning of completion

The frozen V2 engine activated orders at the next book update after their modeled arrival and made cancellations effective immediately. A later audit also identified snapshots stamped before a blocking REST request completed. These details could change fill eligibility and queue age. The project froze the old evidence as `legacy_book_update_v1`, implemented `event_driven_v2`, and reran the development experiment in a distinct V3 namespace.

The V3 protocol was committed at `63f4f0b` before its corrected economics were inspected; `04df629` identifies the research source. Completion means executing that protocol, reporting its actual outcome, preserving the holdout boundary, and validating the native work within its stated arithmetic and integration contract. It does not require advancing a candidate that failed the gate or relabeling the historical sensitivity grid as current-model evidence.

2 Data, Reconstruction, and System Architecture

2.1 Recorded data and its clock fields

The empirical study uses Binance spot BTCUSDT depth diffs, periodic REST book snapshots, and aggregate trades. Records are stored as hourly gzip-compressed JSON Lines files. The recorded REST snapshots contain 1,000 levels per side; the subsequent diff stream updates aggregate quantities at named prices. Separate perpetual-futures recordings exist locally but do not enter this experiment.

Depth updates expose a first and last update ID, bid and ask changes, and event time `E`. Aggregate trades expose an aggregate trade ID, price, quantity, matching time `T`, and buyer-maker flag. A zero depth quantity removes a level; a nonzero quantity replaces its displayed size. These field and update semantics follow the exchange’s spot stream documentation [2]. REST snapshots supply `lastUpdateId` but do not supply the exchange timestamp of the represented book [1].

The project retains the difference among local request time, local receipt time, depth event time, and trade matching time. It does not substitute the recorder’s wall clock for trade `T`. The selected replay axis is an exchange-clock proxy with a local request barrier, rather than a calibrated client-observation clock.

2.2 Snapshot bridging and fail-closed recovery

If a snapshot ends at update ID S , diffs with final ID $u \leq S$ are stale. The first retained diff must satisfy the bridge condition $U \leq S + 1 \leq u$; subsequent diffs must maintain sequence continuity. Depth and aggregate-trade IDs are checked independently, including across hourly files. A known gap invalidates the relevant timestamp group and pauses execution until trustworthy snapshot reconstruction is available.

The legacy recorder attached its snapshot tag before a blocking REST request completed. Applying the returned book at that tag could expose information too early. Current captures write request and response times separately. `post_response_proxy_depth_boundary_v1` emits a fail-closed barrier at request time, establishes a valid sequence bridge, and requires a retained depth receipt after response completion. For old captures, the recorder’s blocking behavior makes the first strictly later local depth receipt a defensible upper-bound proxy for response completion. Recovery is released on the selected depth record’s event time E ; only the final reconstructed book is visible to strategy logic or sampling.

The audit of 120 selected development depth files found a valid bridge for every snapshot. The first later receipt followed the old tag by a median 250.5 ms, a 90th percentile of 672.9 ms, and a maximum of 2,335 ms. These are proxy lags, not measured REST round trips. Two zero-credit and four proportional-credit V2 fills came from orders placed before the selected boundary. Those counts do not bound all queue-age consequences, which is why V3 reran the complete panel rather than subtracting a few exposed fills.

2.3 Integrity inventory and selected panel

The frozen inventory covers 1,193 hourly slots before 1 June 2026. Required file availability, gzip and JSON decoding, snapshot bridging, depth sequence, cross-file continuity, and aggregate-trade continuity left 775 eligible hours and 418 invalid hours. A five-hour window containing an invalid hour was ineligible. Most exclusions reflected missing captures; selection need not be independent of market conditions.

The development panel contains 24 non-overlapping five-hour windows: six exploratory anchors and 18 later windows selected from the eligible inventory while balancing UTC buckets and one-second realized-volatility terciles. Development is clustered in April and early May. Twelve late-May windows form the strategy-sealed holdout. Their identities and pre-strategy regime descriptors were examined; their strategy outcomes were not. The descriptors label the holdout regime-shifted, with lower volatility and fewer jumps.

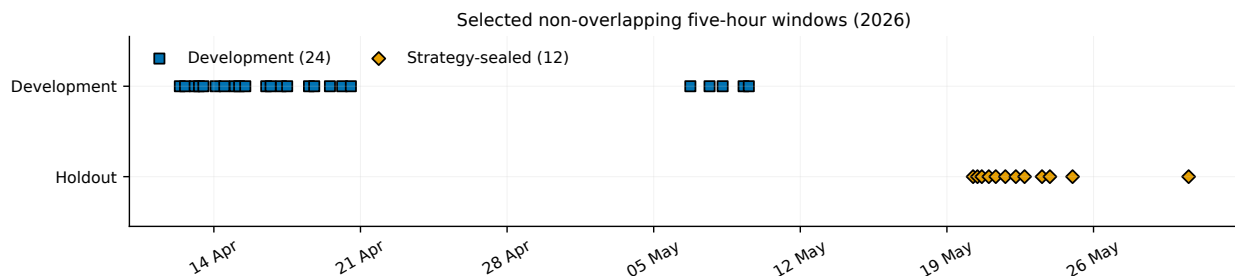


Figure 1: Unchanged frozen selection used for V2 and corrected V3. The figure is retained from the historical report because window membership did not change. Holdout markers identify reserved windows, not evaluated strategies.

The V3 input audit verified all 240 selected spot depth/trade files against their recorded SHA-256 values: 824,383,459 bytes in total. The inventory’s logical identity, the development CSV’s byte hash, and the run’s source hash are distinct objects. They are listed in Appendix A; interchanging them would make a provenance check meaningless.

2.4 Software responsibilities and flow

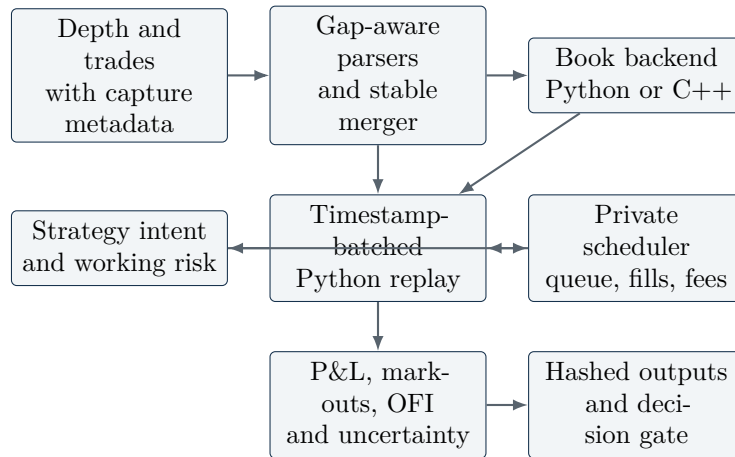


Figure 2: Research flow after native integration. The selected backend owns book state; Python retains parsing, execution, strategy, and research accounting. Native backend equivalence is assessed separately from the frozen Python V3 experiment.

Table 2: The component boundaries make assumptions testable.

Component	Responsibility
<code>src/replay/</code>	Decimal reference book, depth/trade parsers, event merge, timestamp groups, gap policy, and replay termination.
<code>src/execution/</code>	Private-order lifecycle, entry and cancel latency, post-only arrival, queue/FIFO transitions, fills, and fees.
<code>src/strategies/</code>	Symmetric and microprice quoting, blocked OFI-gated logic, tick rounding, requotes, and worst-case exposure.
<code>src/analysis/</code>	Accounting reconciliation, matched lots, forward marks, OFI/microprice regressions, bootstrap intervals, fee and tail diagnostics.
<code>scripts/</code>	Recording, panel orchestration, protocol validation, source/input identity, artifact verification, and report generation.
<code>cpp/</code>	Checked native book, canonical serialization, hashing, command-line interface, Python binding, and native tests.

Strategies request actions; they do not decide whether those actions have arrived or filled. The execution simulator owns those transitions. Decimal accounting and seeded independent entry/cancel randomness preserve repeatable behavior. Canonical book hashes expose reconstruction divergence, while event and accounting comparisons are needed to expose execution divergence. These are complementary checks.

3 Deterministic Execution and Risk

3.1 The execution model defines the fill sample

Arrival, cancellation, and queue rules determine which recorded trades can fill a hypothetical order. They therefore determine both economic outcomes and the conditioning set for the signal study. V3

uses `event_driven_v2`, `market_data_before_private_actions_v1`, `post_response_proxy_depth_boundary_v1`, and the `pause_until_snapshot` trade-gap policy. V2 economics remain labeled `legacy_book_update_v1` and are used only for an explicit historical comparison.

Table 3: Behavioral changes that required a new research namespace.

Mechanism	Frozen V2	Corrected V3
Order arrival	Activated at a subsequent depth update	Scheduled at its modeled millisecond
Cancel effect	Immediate at request	Separately delayed; order can fill in flight
Same-ms private events	Implicit through book activation	Entire recorded market group processed first
Own queue	Weaker separation of public and own quantity	Non-overlapping FIFO ranks and conserved trade budget
Snapshot release	Returned state applied at pre-fetch tag	Request barrier and post-response receipt gate
Working risk	Needed stronger cancel/replace envelope	Active, pending, and cancelling orders count together

3.2 Clock and order lifecycle

Private actions strictly before the next market timestamp execute first. At the same millisecond, all recorded depth/trade events execute before private arrivals; unresolved market-data ties use depth before trade and preserve within-stream order. Consequently a new order arriving at a trade’s millisecond cannot capture that trade, and a cancel arriving at the same millisecond can lose the race. This is a deterministic policy for unresolved timestamps, not knowledge of the venue’s sub-millisecond sequence.

For submission at t_s , entry latency L_e , and seeded jitter J_e ,

$$t_a = t_s + L_e + J_e. \quad (1)$$

For a cancel requested at t_r with its own latency and jitter,

$$t_c = \max(t_r + L_c + J_c, t_a). \quad (2)$$

Pending entry contributes potential inventory but cannot fill. At arrival, the simulator checks whether the order would cross. Post-only crossing orders terminate with an explicit reason. Resting orders remain fillable until their cancellation reaches the modeled exchange.

A tested race. A synthetic order is placed at 0 ms, arrives and joins the queue at 5 ms, receives a cancel request at 6 ms, fills at 10 ms, and emits `cancel_too_late` when the cancel arrives at 11 ms. The demo tests the order lifecycle; it does not calibrate the latency assumption to a live venue.

3.3 External queue credit and own-order FIFO

The simulator decomposes an order’s modeled quantity ahead into external displayed volume and re-maining earlier own orders at the same side and price:

$$Q_i^{\text{ahead}} = Q_i^{\text{external}} + Q_i^{\text{own}}. \quad (3)$$

The external queue starts from displayed quantity, with an ordering floor imposed by earlier own orders. A depth decrease unexplained by recorded trades can reduce only the external component:

$$Q_{i,t+1}^{\text{external}} = Q_{i,t}^{\text{external}}(1 - c\rho_t), \quad c \in [0, 1], \quad (4)$$

where ρ_t is the inferred fraction removed and c the queue-cancellation credit. At $c = 0$, unexplained cancellations confer no advancement; at $c = 1$, the model gives proportional advancement. Neither endpoint is observed ground truth, and they are not guaranteed mathematical bounds on actual execution.

Depth-decrease attribution waits until all trades at the same millisecond have been observed. Otherwise depth-before-trade processing could mistake traded quantity for cancellation and award improvement twice. Recorded aggressor volume advances non-overlapping own FIFO ranks. The complete transition is planned and validated before mutation, with the invariant

$$\sum_i q_{i,t}^{\text{maker fill}} \leq q_t^{\text{recorded aggressor}}. \quad (5)$$

Cancelling an earlier own order releases its remaining own quantity from later orders' ranks. A scale-aware Decimal tolerance of 10^{-24} handles division residue; it cannot create fill volume. Partial fills preserve remaining order quantity and FIFO relationships.

3.4 Worst-case working exposure

Inventory alone understates risk during delayed cancel/replace. Let I_t be current inventory and B_t, S_t the remaining quantities of all buy and sell orders that can still fill. These sets include active orders, pending entries, and live orders with cancels in flight. The admission envelope checks the appropriate prospective side against

$$I_t + \sum_{i \in B_t} q_i \leq I_{\max}, \quad I_t - \sum_{i \in S_t} q_i \geq -I_{\max}. \quad (6)$$

Opposite-side working quantity is not assumed to offset a same-side fill scenario. Breaching an invariant aborts replay rather than being recorded as a temporary harmless overshoot. The experimental cap is 0.01 BTC, and each new baseline quote is 0.001 BTC.

3.5 Accounting, marks, and termination

Let fill direction s_i be +1 for a buy and -1 for a sell, quantity q_i , price p_i , and fee f_i . With cash C and inventory I ,

$$C \leftarrow C - s_i p_i q_i - f_i, \quad I \leftarrow I + s_i q_i, \quad V_t = C_t + I_t m_t. \quad (7)$$

Maker/taker fee classification follows actual modeled execution. The baseline uses post-only orders and produced no taker fills in V3, but the accounting code still distinguishes the two paths. Reconciliation separates completed lots from residual inventory so matched-lot statistics are not confused with full-strategy marked P&L.

A known data gap invalidates the complete timestamp group before it can drive fills or strategy callbacks. Local order and queue state is invalidated until snapshot recovery. This is fail-closed research bookkeeping; it does not assert that exchange orders disappear on a feed failure. At the last market event, private actions scheduled later are not executed against an invented future book. Remaining live orders expire. Pending-action diagnostics are captured before that final cleanup and can therefore be nonzero.

Each nominal five-hour result sums five independently initialized hourly episodes. Inventory is marked at each hour's final mid and reset without liquidation cost. The resulting statistic measures repeated short episodes, not one continuous five-hour portfolio or deployable account return.

4 Locked Research Design

4.1 Price, microprice, and order-flow imbalance

For best bid b_t , best ask a_t , and their displayed quantities q_t^b, q_t^a , the arithmetic mid and microprice are

$$m_t = \frac{b_t + a_t}{2}, \quad \mu_t = \frac{q_t^b a_t + q_t^a b_t}{q_t^b + q_t^a}. \quad (8)$$

The microprice moves toward the ask when bid-side displayed quantity is larger. Its deviation from mid is expressed in basis points. This is the project's simple reference-price signal, not an optimal-control solution calibrated to an arrival-intensity model.

Top-of-book order-flow imbalance (OFI) uses the event construction associated with Cont et al. [3]:

$$e_n = \mathbf{1}_{b_n \geq b_{n-1}} q_n^b - \mathbf{1}_{b_n \leq b_{n-1}} q_{n-1}^b \quad (9)$$

$$- \mathbf{1}_{a_n \leq a_{n-1}} q_n^a + \mathbf{1}_{a_n \geq a_{n-1}} q_{n-1}^a. \quad (10)$$

One-second raw OFI sums these increments over the preceding second. Normalized OFI divides the sum by mean displayed bid-plus-ask top depth over contributing updates. The exact implementation is `src/analysis/of_i_signal.py`. The prior literature motivates this observable; the BTCUSDT results here are the project's own measurements.

On the one-second sampling grid, the population response at horizon h is

$$r_{t,h} = 10^4 \frac{m_{t+h} - m_t}{m_t}. \quad (11)$$

Regression reports the signal coefficient, its HAC statistic, R^2 , and predicted drift for a one-standard-deviation signal change. The last quantity makes effect size interpretable across signal scales.

For maker fill i with direction $s_i \in \{-1, +1\}$, the conditional response is

$$z_{i,h} = 10^4 s_i \frac{m_{i+h} - m_{i-}}{m_{i-}}, \quad x_i = s_i \text{OFI}_{i-}. \quad (12)$$

Here m_{i-} and the OFI anchor are strictly before the fill. A book sample at the fill millisecond is excluded because it may already include the move associated with that fill. This response measures side-aligned future mid movement; a separate transaction-price markout measures spread capture and is not substituted into the gate. Maximum anchor staleness and permitted future sample lag are each 1,000 ms.

4.2 Frozen baseline and episode construction

Table 4: V3 parameters locked before the rerun. Latency and fees are model inputs.

Setting	Value
Instrument and strategy	Binance spot BTCUSDT; existing MicropriceMM
Order size and position cap	0.001 BTC per quote; 0.01 BTC maximum position
Half-spread and tick	2.00 USDT from microprice; 0.01 USDT tick
Requote interval	5,000 ms
Entry and cancel latency	10 ms each; zero jitter; seed 42
Maker and taker fee	2 bps and 5 bps; post-only enabled
Queue-cancellation credit	$c = 0$ and $c = 1$
Panel and episode unit	24 five-hour windows; five independent hourly episodes each
Terminal inventory	Mark to hour-end mid, then reset; no liquidation cost

The 2 USDT half-spread and five-second requote were chosen in earlier exploration. The six anchors remain development observations. V3 neither reselects windows nor tunes a threshold after its corrected outcomes. Its purpose is remeasurement of an existing hypothesis under corrected mechanics, not an untouched replication of a randomly chosen baseline.

4.3 Prospective hypotheses and decision rule

The repository protocol `v3_development_protocol_v1` was committed at `63f4f0b`. It predicted a positive pooled one-second OFI coefficient and a selected 30-second conditional contrast below 1.0 bps at both queue endpoints. This is a repository specification, not an external registration.

At each endpoint, population support requires a common coefficient sign in at least 75% of usable windows, pooled HAC $|t| \geq 2$, and an absolute effect of at least 0.05 bps per signal standard deviation. The historical sign-stability rule admits either dominant sign; the positive pooled-coefficient hypothesis is reported separately.

The conditional analysis fixes bucket edges $[-1, -0.25, 0, 0.25, 1]$ for the side-aligned normalized signal. It selects the most-populated negative bucket and the most-populated nonnegative bucket in pooled buy/sell observations, breaking count ties by the existing ascending bucket order. The separation is

$$\Delta = \bar{z}_{\text{selected nonnegative},30s} - \bar{z}_{\text{selected negative},30s}. \quad (13)$$

With at least 30 observations in each selected bucket, $\Delta \geq 1.0$ bps passes and a smaller contrast fails. Missing or thinner buckets are inconclusive. The compatibility label `inconclusive_power` denotes a minimum-count rule, not a power calculation. The selected pair need not be the two extreme buckets, so the entire response curve must accompany the contrast.

Both endpoints must be supported for the combined screen to be supported. Any blocked endpoint blocks the combined screen; otherwise any inconclusive endpoint makes it inconclusive. A five-second fallback is exploratory and cannot replace this primary decision. Even a supported premise would require a separately locked development candidate; it would not automatically authorize holdout evaluation.

4.4 Uncertainty and the unit of inference

Baseline confidence intervals resample the 24 five-hour windows 10,000 times with seed 7 and use two-sided percentile 95% limits. Window resampling preserves dependence within each observed cluster. Session- and matched-lot intervals remain diagnostics because their independence assumptions are

stronger. Resampling practice follows the bootstrap framework [4]; it does not make these mechanically selected market windows a random sample of all regimes.

The population regression uses Newey–West standard errors [5], with the larger of the overlapping-return horizon lag and sample-size lag rule, capped at $n - 1$. This addresses the specified local serial dependence. It does not produce uncertainty across independently sampled market regimes; a window-clustered population-effect interval is not part of the completed protocol.

V3 adds a diagnostic clustered interval for the conditional contrast. The original pooled sample selects the two labels once. Every bootstrap draw resamples all 24 windows with replacement, bringing all their fills and retaining zero-count clusters. Each draw recomputes pooled fill-weighted means for those fixed labels. There are 10,000 draws using Python `Random(7)`. Any draw lacking a selected bucket would withhold the interval rather than be silently discarded. Both endpoints have zero undefined draws.

This interval conditions on the original bucket selection, is not selection-adjusted, assumes independent window clusters, and does not change the gate. Its role is to show how the selected contrast varies across this development panel, not to turn a failed heuristic screen into a universal negative result.

4.5 Holdout boundary

The 12 late-May holdout windows are strategy-sealed. Their membership and pre-strategy regime descriptors are public; no candidate has been evaluated on them. The completed V3 gate blocks the proposed `OFIGatedMM` before candidate or holdout strategy evaluation. A new hypothesis would require a new development protocol and a locked candidate before one holdout evaluation. Leaving the holdout sealed is the prescribed outcome of this experiment.

5 Corrected V3 Results and Historical Context

5.1 Baseline economics under the corrected model

All 61 specified workflow steps completed, producing 330 durable artifacts under the frozen V3 research source. The headline endpoint estimates are in Table 5. Full-precision values, source/output identities, endpoint diagnostics, and the recomputed gate are retained in `results/panels/btcusdt_12_panel_v3_development_summary/summary.json`.

Table 5: V3 baseline and primary conditional results. P&L is an aggregate of reset hourly episodes; the conditional interval is diagnostic.

Measurement	No credit, $c = 0$	Proportional, $c = 1$
Mean net P&L, USDT/window	−0.774	−0.978
95% window-bootstrap interval	[−1.954, +0.182]	[−2.204, −0.081]
Maker fills / taker fills	1,609 / 0	2,030 / 0
Orders submitted	103,871	104,688
Cancels that arrived too late	7	8
30s conditional contrast, bps	−0.456	−0.064
Diagnostic clustered 95% interval, bps	[−1.497, +0.518]	[−0.904, +0.692]
Primary endpoint screen	Blocked	Blocked

Both baseline sample means are negative. The no-credit interval crosses zero; the proportional-credit interval lies below zero. More modeled queue advancement produces more maker fills but does not produce a positive sample mean. These endpoint results do not demonstrate stable profitable execution for the locked baseline.

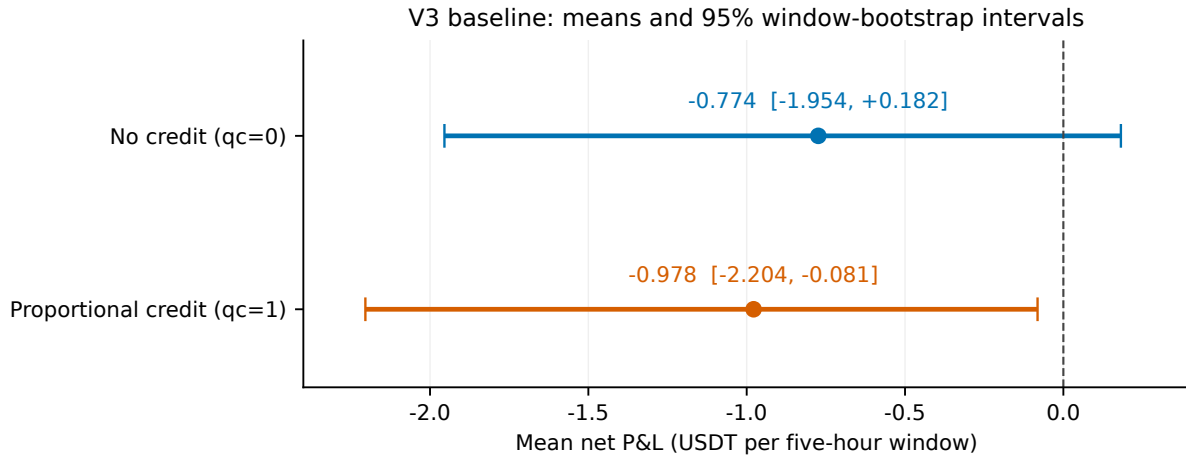


Figure 3: Current V3 mean net P&L and 95% window-bootstrap intervals. Error bars resample windows; they are not intervals for the current-minus-legacy difference.

No selected V3 episode reports a replay gap or invalidation under the frozen eligibility rules. Both endpoints report eight pending private actions and four pending cancellations at the end-of-input diagnostic point, before final expiration. Reporting those fields as zero would erase the explicit termination boundary.

Completed-lot diagnostics also show fees matter. Mean gross and net P&L per matched lot are approximately $+0.002938 / -0.004877$ USDT at $c = 0$ and $+0.002379 / -0.005026$ USDT at $c = 1$. These lot means describe completed fragments with different sizes; they do not replace full-strategy marked P&L or a volume-weighted return. At a representative BTC price of 71,500 USDT, a 2 USDT half-spread is roughly 0.28 bps per side, materially smaller than the modeled 2 bps maker fee. That arithmetic illustrates why fill quality and fees must accompany quoting-price intuition.

5.2 Population OFI survives the correction

The pooled one-second normalized-OFI effect is $+0.123285$ bps per signal standard deviation, with HAC $t = 64.6214$, $R^2 = 0.0365623$, and 430,259 usable population samples. All 24 development-window coefficients are positive. The sign, HAC, and size thresholds all pass.

The population sample is the same at both queue endpoints because queue credit changes hypothetical execution rather than the recorded market path. The positive pooled-coefficient prospective hypothesis therefore holds. The large HAC statistic describes a persistent association in this sample; the low R^2 and short clustered observation period prevent treating it as a broad claim about predictable returns or deployable profits.

5.3 The passive-fill screen fails

At both endpoints, the rule selects the negative bucket $x < -1$ and the nonnegative bucket $0 \leq x < 0.25$. Their counts and mean side-aligned 30-second mid moves are:

Table 6: The actual selected conditional buckets, pooled across buys and sells.

Queue credit	Selected bucket	Maker observations	Mean move, bps
$c = 0$	$x < -1$	685	-1.747
$c = 0$	$0 \leq x < 0.25$	157	-2.203
$c = 1$	$x < -1$	817	-1.810
$c = 1$	$0 \leq x < 0.25$	196	-1.875

Both selected buckets exceed the minimum count, so the endpoint result is `fail_signal`, not `inconclusive_power`. Their contrasts, -0.456 and -0.064 bps, are below the predeclared $+1.0$ bps bar. Neither endpoint invokes the optional five-second fallback.

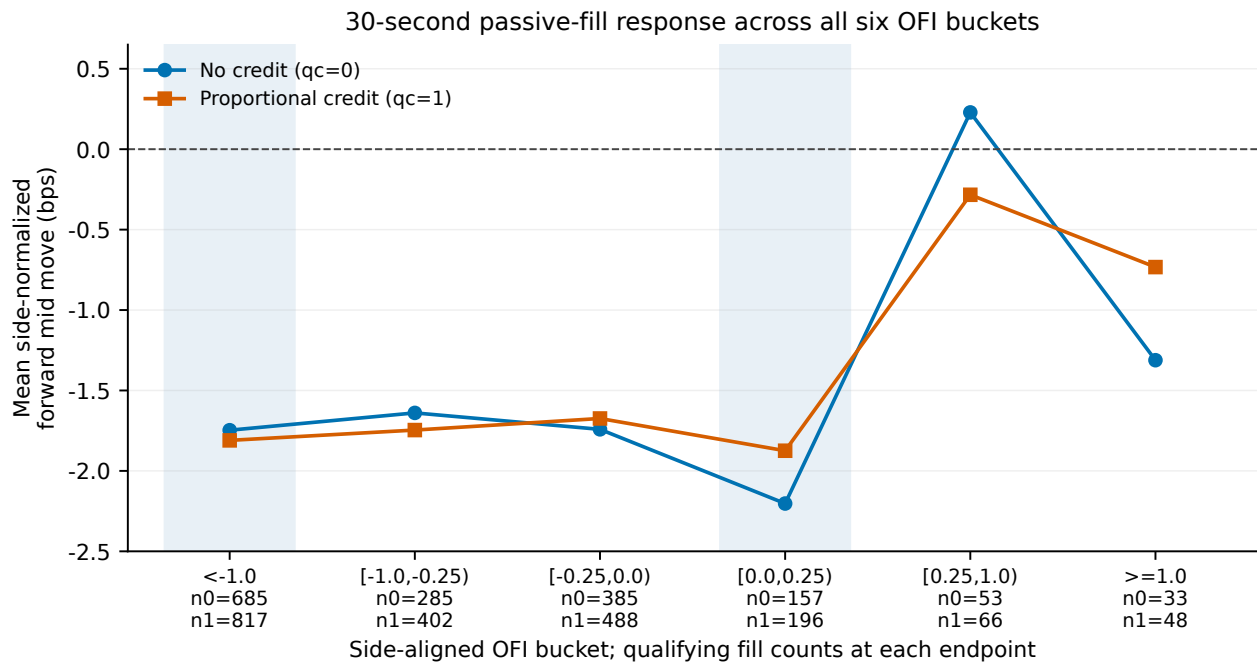


Figure 4: V3 30-second fill-conditioned response across all side-aligned normalized-OFI buckets. Counts accompany the means. The gate selects the most populated negative and nonnegative buckets, not the two extreme buckets; the selected buckets are shaded blue. The full response does not support a simple monotone interpretation.

Every window contributes observations to each selected bucket at both endpoints. The clustered intervals are $[-1.497, +0.518]$ bps at $c = 0$ and $[-0.904, +0.692]$ bps at $c = 1$, with no undefined resamples. They cross zero and condition on the originally selected labels. They support reporting a weakly separated selected comparison, without proving a universal absence of OFI information or the profitability of an unrun transformation.

Decision. The combined development screen is `blocked`. The baseline's passive fills do not clear the defined conditional materiality screen despite population OFI support. No OFI-gated candidate strategy was evaluated, no candidate parameters were tuned, and the holdout remains strategy-sealed.

5.4 What changed relative to historical V2

The old and current studies use the same selected development windows, but different execution and snapshot semantics. Their no-credit mean net P&L changes from -0.686 to -0.774 USDT/window, a descriptive difference of -0.088129 . Proportional credit changes from -1.043 to -0.978 , a difference of $+0.064866$. The no-credit interval still crosses zero and the proportional interval remains below zero. These differences are model sensitivity, not paired confidence intervals or proof that either model is closer to actual exchange fills.

Historical conditional OFI separation was -0.376 bps with no credit and $+0.127$ bps with proportional credit. Both already failed the screen. V3 changes the numeric contrasts and execution composition while preserving the predeclared block. The historical microprice population test also missed its significance and size screens: its one-second effect was about 0.0190 bps per standard deviation, with HAC $t = 1.74$.

V2 additionally swept queue credit $\{0, .25, .5, .75, 1\}$ at 10 ms entry latency and the two endpoints at 0 and 50 ms. That frozen grid showed more modeled credit increasing throughput while matched net economics worsened from roughly -9.75 to -23.65 USDT/BTC. The intermediate-credit and latency grid has *not* been relabeled as V3: the completed current protocol covers the two endpoints at 10 ms entry and cancellation latency. Extending that grid would be a separate versioned experiment.

5.5 What the evidence answers

The corrected measurement supports both prospective statements: the pooled OFI coefficient remains positive, and the defined conditional separation remains below 1.0 bps with adequate selected counts at both endpoints. The useful distinction is between a population association and the conditional question posed by this baseline's fills. It does not establish that passive market making is impossible, that every signal transform fails, or that a blocked candidate would necessarily lose. Those would require additional, separately specified experiments.

6 C++ Design, Integration, and Performance

6.1 Porting against a behavioral reference

The native work began with a bounded contract: reproduce the Python book's state transitions and canonical bytes on the fixed-eight-decimal BTCUSDT input domain. The June golden vectors were recovered unchanged from commit 578b524. A library and standalone CLI established operation and transcript parity before the native book was connected to the Python pipeline. The integration then addressed the remaining gap between a fast isolated component and a usable backend for replay, execution, and analysis.

The result is a hybrid implementation. C++ owns price-level storage and book mutation; Python owns parsing, scheduling, strategy, execution, and research accounting. This preserves the tested Decimal arithmetic for microprice, queue fractions, fees, and P&L while permitting native storage to participate in the entire application path. It does not imply that the whole simulator has been translated into C++.

6.2 Checked fixed-point representation

Each book side is a `std::map<int64_t, int64_t>`. Prices and quantities use the scale 10^8 ; for example, 74574.79000000 is stored as 7457479000000. Direct digit parsing avoids conversion through binary floating point. Ordered maps provide insertion, replacement, and deletion in $O(\log N)$ and deterministic best-price and level traversal. RAII containers own the book memory.

The parser requires unsigned decimal strings with exactly eight fractional digits, checks scaled-integer overflow, and validates tick/quantity-step multiples. Stored prices and nonzero quantities must be at least 0.00000100 , where the reference Decimal canonical representation stays in ordinary fixed-point

notation. Signed strings, other Decimal spellings, and values outside this domain are rejected. Tick and quantity-step metadata can be normalized exactly by the Python adapter; incoming levels must still obey the native contract.

Snapshots replace both sides and skip zero quantities. Diffs replace nonzero quantities and erase zero levels, including absent levels without error. Duplicate prices retain the reference input-order behavior. Crossed books remain representable. Both sides of an operation are validated before a rejected input can mutate state. Allocation failure during an otherwise valid diff is terminal; the contract does not promise that the same book can be reused after `std::bad_alloc`.

Canonical JSON preserves Python's key order, price order, eight-decimal level strings, and update ID. OpenSSL EVP computes SHA-256. A transcript digest hashes the ASCII state hash after every operation, including the initial empty state. Comparing only the final book could miss divergence that later reconverges; the transcript cannot conceal an unequal intermediate state.

6.3 The Python/native boundary

`cpp/src/python_module.cpp` supplies a CPython extension without a second binding framework. A capsule owns one C++ book and releases it with Python object lifetime. Exceptions map to Python errors. Calls retain the GIL; the binding is not a mechanism for parallel execution within one Python process.

`src/replay/cpp_orderbook.py` implements `CppOrderbook` against the reference book interface. It keeps no mirrored `SortedDict`. After an update, one native return refreshes cached best bid/ask prices and quantities, counts, update ID, and sequence. Those public price and quantity values become Python `Decimal` objects. Inherited mid, spread, and microprice formulas therefore execute with the same arithmetic as the reference. Price-level quantity and ordered-level accessors query native storage for execution logic and diagnostics.

The shared backend factory selects Python by default and C++ explicitly. `--book-backend cpp` is propagated through replay, comparison, reconciliation, microprice and OFI analysis, and panel orchestration. A missing or invalid native module fails clearly; it does not silently select Python. The loader checks the extension API and hashes the loaded binary. Backend identity, instrument-grid metadata, native build identity, and reference source identity travel with provenance and cache keys. Native outputs use a separate namespace so a backend experiment cannot overwrite frozen V3 research artifacts.

This division is intentional. Reimplementing proportional queue arithmetic or P&L using rounded native fixed-point operations would introduce a new behavioral problem before proving storage equivalence. The present boundary keeps derived arithmetic unchanged and measures the conversion overhead it actually incurs.

6.4 Layered parity evidence

Native CTest checks decimal boundaries, overflows, invalid-input rejection, snapshot/diff behavior, and hashing. Address/undefined-behavior sanitizer checks exercise the native implementation. Differential tests compare the unchanged golden operations, seeded random traces containing deletes, duplicates, resets and crossed states, and every canonical state in a real development-hour trace.

The isolated recorded-hour test applies 35,999 operations and matches all 36,000 states, including the empty book. Its transcript digest is

```
eec2fd20873bc2105d342ca3b5fe5736941007fcd087eb3eeb6c4140e5fd8731
```

The final state hash is

```
8acb43778de91b3029ed91e83280a9c4429146303257f5a7be45ec9d54f12dff
```

Pipeline validation goes beyond equal book hashes. The differential runner compares complete

streams of checkpoints, order events, fills, book samples, markouts, queue diagnostics, population and fill-conditioned OFI samples, plus hold time, P&L, statistics, final inventory, fees, and final book. Backend identity may differ; execution semantics may not. Inputs are accepted only from the hash-matching frozen development inventory. Holdout data is excluded from this engineering validation.

The first recorded pipeline comparison exposed why this extra layer matters: an absent price-level lookup became Decimal $0\text{E}-8$ in the adapter while the reference returned Decimal 0. Those values are numerically equal, but their string representations differed in order-event and queue-diagnostic streams. Returning the reference’s canonical zero corrected the discrepancy. Final book equality or a P&L-only comparison would not have caught it.

Native storage matched Python across all 120 selected development hours and 240 hour/endpoint runs. The comparison hashes every order event, fill, book sample, markout, queue diagnostic, OFI population and conditional sample, and final accounting state. It also checks hold-time and P&L summaries, replay statistics, and book checkpoints every 1,000 market events. All hourly replay statistics, net P&L, fees, and final positions also match the frozen V3 reconciliation artifacts. No holdout strategy outcomes were evaluated.

Queue endpoint	Python median (s)	Native median (s)	Ratio
No credit	5.638	4.826	1.168 ×
Proportional credit	5.672	4.771	1.189 ×

Table 7: Integrated pipeline measurements for 12 April 2026, 09:00 UTC. One warmup and three measured repetitions per backend, with alternating backend order and a warm operating-system page cache. Ratios compare the measured medians and need not indicate an overall speedup.

These timings include raw gzip reads and JSON parsing, object construction, replay, strategy callbacks, execution, checkpoint hashing, book samples, markouts, P&L, hold-time and queue diagnostics, and OFI sample construction. They exclude imports and initial native discovery, parity serialization, object destruction, output writing, pooled regressions, bootstrap inference, and report generation. Every measured repetition is checked against the Python output streams. This is a separately scoped measurement from the preloaded book-update benchmark.

6.5 Isolated book-update measurement

The standalone benchmark first verifies transcript parity and then applies identical preloaded operations. It times book construction, dispatch, decimal conversion, and updates. Decompression, JSON/protocol parsing, state hashing, process startup, book destruction, callbacks, and execution are outside the timed region. Python book destruction was initially capable of falling inside the next iteration’s timer; commit `fc07501` corrected that boundary before the following measurements were accepted.

Table 8: Standalone book-update timings on 11 September 2026. Intel Core i5-12400F; Python 3.11.14; GCC 13.3.0 Release, `-O3 -DNDEBUG`; one warmup and five measured repetitions. Ratios compare medians.

Input	Python median	C++ median	Ratio
Synthetic, 20,000 updates	85.445115 ms	6.892484 ms	12.39685 ×
Recorded hour, 35,999 updates	623.006815 ms	45.326177 ms	13.74497 ×

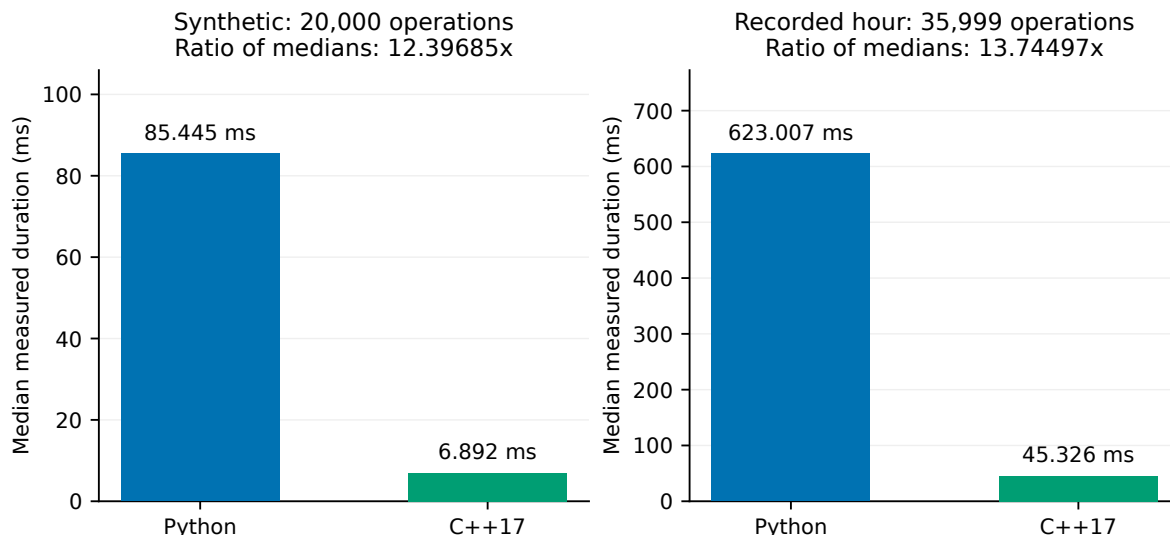


Figure 5: Parity-gated standalone book-update measurements. These ratios exclude the CPython adapter and the rest of replay; they must not be reported as full-application acceleration.

The synthetic input has seed 42 and 200 initial levels per side. The recorded input is the depth hour beginning 12 April 2026 at 09:00 UTC. Python was timed first and C++ second; all repetitions, environment descriptors, source and binary hashes, and input identity are retained. These are local performance measurements, not a general hardware-independent factor.

6.6 Integrated timing has a different denominator

`scripts/validate_native_pipeline.py` separately times construction, reopened raw gzip/JSON inputs, replay, strategy, execution, checkpoints, book sampling, markouts, P&L, hold-time and queue diagnostics, and OFI sample construction. Initial imports/module discovery, parity serialization, destruction, output writing, pooled regression, bootstrap, and report generation are excluded. Timed repetitions alternate backend order, retain each duration, and rerun parity against the reference before accepting a measurement. Reopened inputs are normally served from the warm operating system page cache.

This broader path includes Python/native conversion and work unaffected by native storage. Even a large isolated update ratio need not translate to an application speedup. Reporting a slower or similar integrated result is an engineering finding about the chosen boundary, rather than a reason to substitute the isolated microbenchmark for application evidence. The binding provides a usable native backend; its measured benefit depends on the workload.

7 Verification, Failure Analysis, and Limits

7.1 What verification establishes

The project uses complementary forms of evidence. Synthetic tests isolate mechanics such as post-only arrival, cancel/fill races, same-millisecond ordering, gap invalidation, volume conservation, and worst-case exposure. Artifact tests check identities, workflow completeness, expected paths, parameter locks, and decision rules. Differential tests check book and pipeline behavior across implementations. Native tests and sanitizers add memory and arithmetic checks. None of these alone establishes that aggregate L2 queue assumptions match a live exchange.

The V3 manifest lists every expected workflow step, its command and durable outputs, source identity, and raw-input hashes. The verifier derives the complete expected 61-step workflow and rejects

missing steps, redirected output paths, incomplete artifacts, and mismatched frozen inventory entries. A self-consistent empty directory is not accepted as a completed study. The decision summarizer recomputes primary gates from the underlying regression and bucket results and independently reconstructs clustered contrasts.

Recorded-revision verification resolves the manifest’s exact Git commit and hashes its Python source bytes. It does not accept an arbitrary commit merely because a subset of files happens to match. Default verification instead requires the current working source to match the research fingerprint. This distinction allows later native integration, benchmark corrections, and verification improvements without rewriting the historical research run’s identity.

Clone-level `--artifacts-only` verification checks committed artifacts, the frozen input inventory, and recorded source identity without claiming to have read absent raw captures. Full verification additionally hashes the selected raw files. Both modes state their boundary explicitly. CSV byte identity is protected against checkout line-ending conversion, because line-ending changes can invalidate a correct logical table’s recorded hash.

7.2 Failures that changed the project

Table 9: Material failures and their disposition.

Failure	Consequence	Disposition
Bar timing and annualization	Same-close fills and incorrect frequency scaling made historical ratios too strong.	Phase 1 retained as educational evidence; metrics helper corrected without rewriting old results.
Accidental taker execution	Intended maker limits crossed after latency, contaminating attractive P&L.	Post-only arrival checks terminate crossing orders and preserve explicit reasons.
Stale snapshot diffs	Replayed stale updates could roll the book backward.	Stale rejection, valid bridge requirement, cross-file continuity, and regression tests.
Pre-fetch snapshot tag	Returned state could be exposed before response completion.	Request barriers, post-response receipt gate, 120-hour proxy audit, and a full corrected V3 rerun.
Missing aggregate trades	Incomplete aggressor volume corrupted execution eligibility and queue estimates.	Fail-closed timestamp invalidation and snapshot recovery.
Same-ms signal leakage	Fill-time book state could contain the price move associated with the fill.	Strictly-pre-fill OFI and mid anchoring, with an explicit leakage test.
Private-event quantization	Arrivals waited for depth updates and immediate cancels omitted races.	Explicit entry/cancel scheduling; legacy/current model and result namespaces separated.
Overlapping own orders	Reused trade volume or incomplete working exposure could manufacture fills or risk capacity.	Non-overlapping own FIFO, conserved fill budget, and active/pending/cancelling exposure checks.

Failure	Consequence	Disposition
Line-ending identity drift	A committed development CSV's LF bytes no longer matched its older CRLF pin.	Pin the actual frozen committed bytes and test the runner's default input; no window reselection.
Incomplete manifest acceptance	A self-consistent partial result tree could appear complete.	Derive all expected workflow commands and paths and require the exact durable set.
Asymmetric benchmark timer	Python book destruction could enter the next measured iteration.	Move destruction outside timing, test the boundary, and replace the affected measurements.
Standalone/native scope gap	Isolated book parity did not establish usability inside replay.	Add a compiled Python binding, explicit backend propagation, whole-stream differential validation, and separate integrated timing.

The common lesson is to correct the measurement boundary before strengthening the claim. Some failures require revised implementation tests; others change the economic estimand and require a new run. A provenance boundary prevents the first kind of fix from quietly relabeling the second kind of evidence.

7.3 Empirical and inferential limits

- **Narrow market coverage.** One venue, one spot instrument, and a short April–May 2026 period cannot establish cross-asset, cross-venue, or long-run robustness.
- **Selected availability.** Only 775 of 1,193 inventoried hours passed the capture requirements. Operational failures can correlate with stressed markets, making missingness economically relevant.
- **Exploratory anchors.** Six baseline-selection windows remain in the 24-window panel. The expanded study is development evidence, not a wholly untouched confirmatory sample.
- **Hourly resets.** Residual inventory is marked and discarded without liquidation cost at each hour boundary. Results are not continuous portfolio returns or a capacity study.
- **Dependence.** Window bootstrap and local HAC errors answer different uncertainty questions. Nearby windows can share regimes; neither method creates independent market histories.
- **Conditional selection.** The clustered contrast interval fixes bucket labels selected from the pooled sample and is not selection-adjusted. The 30-observation floor and 1.0 bps screen are heuristics, not a formal power design or a universal profitability criterion.
- **Holdout status.** Reserved identities and regime descriptors were examined, but strategy outcomes remain sealed. There is no unseen candidate result supporting the report's conclusion.

7.4 Execution and operational limits

Aggregate L2 reveals displayed quantity, not individual order rank, hidden liquidity, iceberg refresh, or the location of cancelled orders in the queue. Cancellation credit is uncalibrated. The modeled timestamp axis mixes local request barriers, depth E, trade T, and receipt-order constraints; the snapshot correction removes an identified early-release exposure without calibrating client observation or network latency.

The simulator assumes its small orders do not alter the recorded market. Endogenous market impact, account fee tiers, acknowledgement/fill calibration, exchange throttles, self-trade prevention, and

production gateway behavior are outside the experiment. Counterfactual taker execution uses a shadow liquidity ledger only until the next observed depth update and would require stronger reconciliation for taker-heavy work. No live account calibration is claimed.

The native input domain is deliberately narrower than arbitrary Python Decimal. Its binding retains the GIL and performs Decimal conversion at the boundary, so it does not promise parallel throughput or end-to-end acceleration. Native parity covers implemented semantics on the tested domain; it does not validate queue assumptions or the economic hypothesis.

7.5 Completed outcome and useful extensions

The completed project supplies a deterministic offline workflow, a corrected and source-identified development study, a stopped candidate gate, an intact holdout boundary, a usable native book backend, differential validation, and separately scoped measurements. The negative conditional result is a research outcome, rather than unfinished strategy tuning.

Further work has distinct questions: a new signal transform needs a new development protocol; a current-model credit/latency grid needs a versioned sensitivity study; live calibration needs observed acknowledgements and fills; and a full native execution engine needs a broader arithmetic/event parity contract. Those are extensions of the completed scope, not evidence already supplied by the present results.

A Reproduction and Artifact Map

A.1 Three distinct reproduction levels

A clone contains code, synthetic/golden inputs, selected derived artifacts, and this report's generation inputs. It does not contain the raw market capture. The following levels must be distinguished:

1. **Mechanics and native implementation.** Install the supported Python environment, run deterministic tests, build C++, and compare native behavior on committed/synthetic traces. These do not require raw data.
2. **Committed evidence.** Verify the historical V2 structure and current V3 durable artifacts, re-construct the decision summary from committed outputs, and regenerate this report. Artifact-only mode verifies recorded input identities but does not claim to read missing capture bytes.
3. **Full empirical replay.** Restore all selected raw files at their manifest paths with matching SHA-256 values, use the frozen research source, and rerun the panel. Native pipeline comparisons similarly require selected raw development captures.

The project is therefore artifact-verifiable from its public repository and fully replayable only with the specified captures. A newer download covering the same dates is not an interchangeable raw input: packet order, snapshot timing, and capture gaps are part of the recorded experiment.

A.2 Install and verify mechanics

Python 3.11 is the supported reference environment. From a complete Git clone:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pip install --no-deps --editable ".[analysis,dev]"
env PYTHONPATH=. pytest -q
env PYTHONPATH=. python scripts/demo_event_driven_execution.py
env PYTHONPATH=. python scripts/verify_v2_artifacts.py
```

Native builds require a C++17 compiler, CMake 3.18 or newer, OpenSSL development headers/libraries, and CPython development support for the selected interpreter. The Make target builds the native tests, CLI, and Python binding:

```
make cpp-test PYTHON=python
```

For Conda-provided OpenSSL, pass `CMAKE_ARGS="-DOPENSSL_ROOT_DIR=$CONDA_PREFIX"`. The extension is built for that Python interpreter and is not assumed portable across Python ABIs. The adapter loads the build-directory extension, or the exact module selected by `L2MM_CPP_MODULE`. `L2MM_CPP_BINARY` similarly selects the CLI for standalone parity tests. Explicit selection fails if the requested artifact is absent or invalid.

A.3 Verify V3 without raw captures

The recorded source commit must be available in Git history; shallow clones must first obtain that history. A later tooling commit is expected to differ from the research source, so the historical-source choice is explicit:

```
env PYTHONPATH=. python scripts/verify_v3_artifacts.py \
  --source-revision recorded --artifacts-only
env PYTHONPATH=. python scripts/summarize_v3_development.py \
  --source-revision recorded --artifacts-only
```

Omit `--artifacts-only` when the selected raw captures are present to hash-check those bytes as well. Omit `--source-revision` only when the current working Python source is intended to be exactly the recorded research source. The verifier never rewrites the run manifest to accommodate later code changes.

A.4 Reproduce the frozen research run

The original V3 experiment is identified by the full commit below. An isolated checkout avoids mixing later backend or tooling changes with the frozen experiment:

```
git worktree add --detach ../l2mm-v3-research \
04df6294872a256374f910d12d5e5e3e08e8757a
```

Restore the selected captures under that checkout's `data/raw/` layout using `DATA.md` and the inventory. With the project environment active, the original serial, resumable entry point is:

```
env PYTHONPATH=. python scripts/run_l2_panel.py --phase all
```

The completed research run scheduled independent commands concurrently through the same runner checks; OFI commands sharing a cache remained sequential. Scheduling did not change replay parameters or algorithms. The durable manifest contains all 61 completed steps and 330 output identities. Ephemeral caches and progress files are not substituted for durable evidence.

A.5 Use and validate the native backend

The current checkout exposes `--book-backend cpp` on replay and analysis entry points. Native output and cache identities remain separate from the frozen Python research result. Command help describes the input-specific arguments for each entry point:

```
env PYTHONPATH=. python scripts/run_replay.py --help
env PYTHONPATH=. python scripts/run_l2_panel.py --help
```

Full development pipeline parity compares both queue endpoints on the selected 120 hourly inputs. Worker count controls independent validation jobs, not concurrent timing repetitions:

```
env PYTHONPATH=. python scripts/validate_native_pipeline.py \
--workers 2 \
--output results/native_pipeline/development_parity.json
```

The recorded-hour application measurement is separate from standalone book timing and reruns stream parity before accepting durations:

```
env PYTHONPATH=. python scripts/validate_native_pipeline.py \
--starts 2026-04-12T09:00:00 --benchmark-repeats 3 \
--output results/native_pipeline/development_hour_benchmark.json

make cpp-benchmark PYTHON=python
```

These commands remeasure the local system and may produce different durations from the committed benchmark. Correctness identities are expected to match; wall-clock timings are observations of a particular environment and load.

A.6 Build the complete report

Tectonic 0.17.0 is the pinned TeX engine. The generated figures and numeric macros use committed research and benchmark artifacts. The complete report's targets preserve the historical V2 PDF:

```
make complete-report
make complete-report-check
```

The output is `report/l2_mm_system_complete_report.pdf`; its source is `report/v3/main.tex`. The existing `make report` builds the historical `report/l2_mm_research_report.pdf`. Report checks inspect the generated assets, PDF structure and text, embedded fonts, build diagnostics, and recorded output identity. Poppler supplies `pdftotext`, `pdfinfo`, `pdffonts`, and `pdftotext`; `qpdf` can add a structural check.

A.7 Canonical identities

Table 10: Distinct hashes identify protocol, selection, source, and durable outputs.

Object	Identity
Integrity inventory logical identity	a3a99b0a616abe3bc39e0863ed047f075db9ed5d8118ced57c16140b499d8a61
Selected-panel identity	760c55b7c0929b4a99657f6ca02eb723d930b9f48ebd3786d57bcb1a0f481122
Development CSV SHA-256	c779138fdffb739715c53cfa27c75b3c8ca140bc4f5a6128f60f2251948bd362
V3 protocol SHA-256	e1890d2e4fa715154f28778febe3367d391a8c8be1b44b766b4ccc0decf3e395
V3 research Git commit	04df6294872a256374f910d12d5e5e3e08e8757a
V3 Python source fingerprint	45141ba610bb070217e6d08bbb5fef9ca1a725f5c7d5e40b6e14881450a4cb4d
V3 durable manifest SHA-256	d70195704c98a14f7ce0daf747b97906754d7d2ffebcc181d308dad8af57d27a

The inventory identity is a logical content identity, not the SHA-256 of its JSON file. The source fingerprint hashes the selected Python source tree rather than the compiled extension. Native evidence separately records its source and binary identities. Later integration code does not replace 04df629 as the source of the V3 economics.

A.8 Artifact map

Table 11: Primary files for independent inspection. Paths are relative to the repository.

Path	Purpose
<code>notebooks/v3_development_protocol.md</code>	Prospective hypotheses, frozen parameters, primary gate, bootstrap procedure, and stopping boundary.
<code>results/panels/btcusdt_l2_panel_v2/</code>	Frozen inventory, development/holdout selection, and historical V2 empirical evidence.
<code>results/panels/btcusdt_l2_panel_v3_event_driven/ARTIFACT_MANIFEST.json</code>	Complete V3 workflow, source and raw identities, and durable artifact hashes.
<code>results/panels/btcusdt_l2_panel_v3_development_summary/summary.json</code>	Recomputed decision, both endpoint diagnostics, uncertainty, and descriptive legacy comparison.
<code>results/replay_correctness/snapshot_timing_panel24.json</code>	Selected-depth snapshot timing audit and observed legacy exposure.

Path	Purpose
<code>cpp/parity_contract.md</code>	Frozen native input, operation, serialization, and arithmetic scope.
<code>results/cpp_orderbook/development_hour_parity.json</code>	Every-state recorded depth parity, without performance timing.
<code>results/cpp_orderbook/synthetic_20000.json</code>	Corrected standalone synthetic book benchmark.
<code>results/cpp_orderbook/development_hour.json</code>	Corrected standalone recorded-hour book benchmark.
<code>results/native_pipeline/development_parity.json</code>	Full replay and diagnostic stream comparison at the native backend boundary.
<code>results/native_pipeline/development_hour_benchmark.json</code>	Separately scoped application-path timing with parity gates.
<code>report/v3/generated/asset_manifest.json</code>	Report presentation inputs and generated asset identities.

The root `results/` tree also includes exploratory and superseded outputs. Use `results/README.md`, the versioned protocol, and the canonical manifest before citing a file. An attractive number elsewhere in the tree is not automatically part of the completed study.

References

- [1] Binance. Spot rest api: Market data endpoints. <https://developers.binance.com/docs/binance-spot-api-docs/rest-api/market-data-endpoints>, 2026. Accessed 11 September 2026.
- [2] Binance. Spot websocket market streams. <https://developers.binance.com/docs/binance-spot-api-docs/web-socket-streams>, 2026. Accessed 11 September 2026.
- [3] Rama Cont, Arseniy Kukanov, and Sasha Stoikov. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014. doi: 10.1093/jfinec/nbt003.
- [4] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.
- [5] Whitney K. Newey and Kenneth D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708, 1987. doi: 10.2307/1913610.